SHAQRA UNIVERSITY
College of Computing and Information Technology

Programming Language (2)

CSC 213

Exception Handling in python programming

In this Chapter

- » Introduction
- » Syntax Errors
- » Exceptions
- » Built-in Exceptions
- » Raising Exceptions
- » Handling Exceptions
- » Finally Claus

1.1 INTRODUCTION

Sometimes while executing a Python program,

the program does not execute at all

or

the program executes but generates unexpected output or behaves abnormally.

These occur when there are **syntax errors, runtime errors** or **logical errors** in the code.

In Python, **exceptions** are errors that get triggered automatically.

However, exceptions can be forcefully triggered and handled through program code.

In this chapter, we will learn about **exception handling in Python programs**.

1.2 SYNTAX ERRORS

Syntax errors are detected when we have not followed the rules of the particular programming language while writing a program.

These errors are also known as **parsing errors**.

On encountering a syntax error, the interpreter **does not execute** the program unless we rectify the errors, save and rerun the program.

When a syntax error is encountered while working in shell mode, Python displays the name of the error and a small description about the error as shown in Figure 1.1.

```
>>> marks=10
>>> if marks>20:
        print "GOOD SCORE"

SyntaxError: Missing parentheses in call to 'print'. Did you mean print("GOOD SC
ORE")?
>>> |
```

Figure 1.1: A syntax error displayed in Python shell mode

So, **a syntax error** is reported by the Python interpreter giving a brief explanation about the error and a suggestion to rectify it.

Similarly, when a syntax error is encountered while running a program in script mode as shown in Figure 1.2,

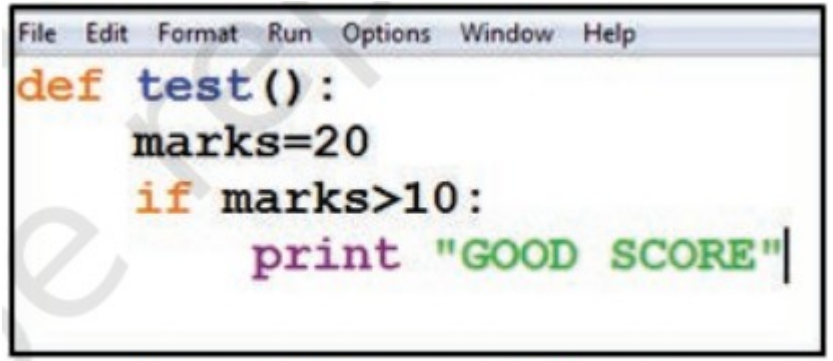


Figure 1.2: An error in the script

a dialog box specifying the name of the error (Figure 1.3) and a small description about the error is displayed.

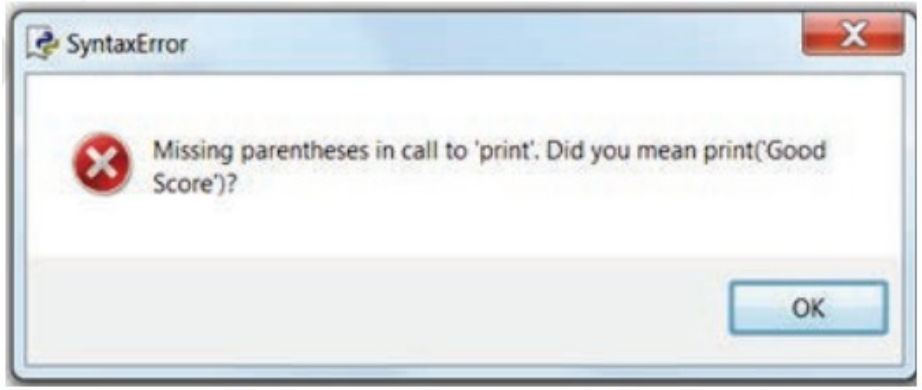


Figure 1.3: Error dialog box

1.3 EXCEPTIONS

Even if a statement or expression is **syntactically correct**, there might arise an error during its execution.

For example,

trying to open a file that does not exist,
division by zero and so on.

Such types of errors might disrupt the normal execution of the program and are called **exceptions**.

1.3 EXCEPTIONS

An exception is a **Python object** that represents an error.

When an error occurs during the execution of a program, an exception is said to have been **raised**. Such an exception needs to be handled by the programmer so that the program does not terminate abnormally.

Therefore, while designing a program, a programmer may anticipate such erroneous situations that may arise during its execution and can address them by including appropriate code to handle that exception.

It is to be noted that **SyntaxError** shown at Figures 1.1 and 1.3 is also an exception. But, all other exceptions are generated when a program is syntactically correct.

```python
try:
    numerator = int(input("Enter the numerator: "))
    denominator = int(input("Enter the denominator: "))
    result = numerator / denominator
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Please enter valid numeric values.")
finally:
    print("Execution complete.")
```

Result 1

```
Enter the numerator: 2
Enter the denominator: 0
Error: Division by zero is not allowed.
Execution complete.
```

Result 2

```
Enter the numerator: e
Error: Please enter valid numeric values.
Execution complete.
```

Result 3

```
Enter the numerator: 1
Enter the denominator: 2
Result: 0.5
Execution complete.
```

1.4 BUILT-IN EXCEPTIONS

Commonly occurring exceptions are usually defined in the compiler/interpreter. These are called built-in exceptions.

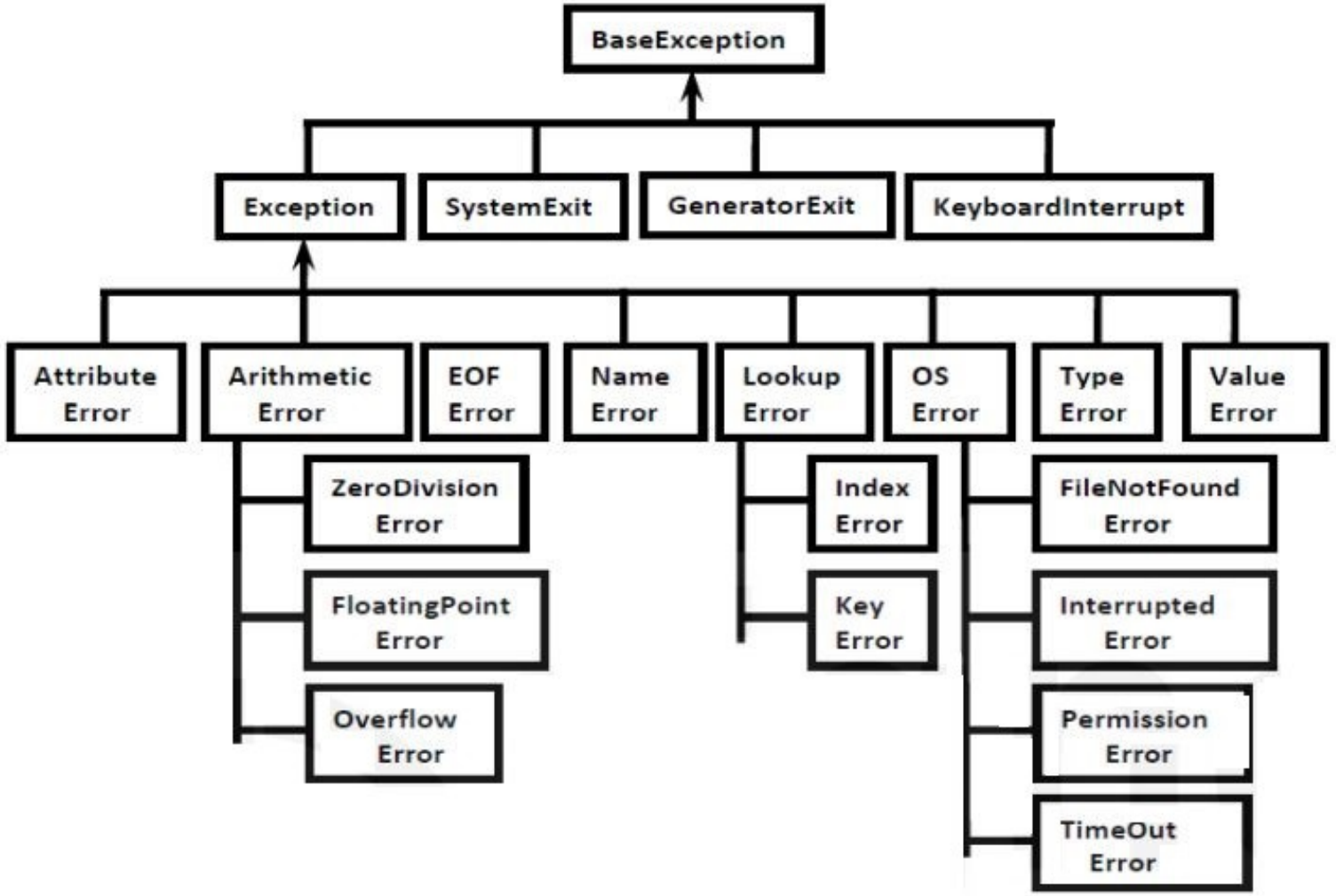
Python's standard library is an extensive collection of built-in exceptions that deals with the commonly occurring errors (exceptions) by providing the standardized solutions for such errors.

On the occurrence of any built-in exception, the appropriate exception handler code is executed which displays the reason along with the raised exception name. The programmer then has to take appropriate action to handle it.

Some of the commonly occurring built-in exceptions that can be raised in Python are explained in Table 1.1

Table 1.1 Built-in exceptions in Python

S. No	Name of the Built-in Exception	Explanation
1.	SyntaxError	It is raised when there is an error in the syntax of the Python code.
2.	ValueError	It is raised when a built-in method or operation receives an argument that has the right data type but mismatched or inappropriate values.
3.	IOError	It is raised when the file specified in a program statement cannot be opened.
4	KeyboardInterrupt	It is raised when the user accidentally hits the Delete or Esc key while executing a program due to which the normal flow of the program is interrupted.
5	ImportError	It is raised when the requested module definition is not found.
6	EOFError	It is raised when the end of file condition is reached without reading any data by input().
7	ZeroDivisionError	It is raised when the denominator in a division operation is zero.
8	IndexError	It is raised when the index or subscript in a sequence is out of range.
9	NameError	It is raised when a local or global variable name is not defined.
10	IndentationError	It is raised due to incorrect indentation in the program code.
11	TypeError	It is raised when an operator is supplied with a value of incorrect data type.
12	OverFlowError	It is raised when the result of a calculation exceeds the maximum limit for numeric data type.



In Python, every exception is class, and these classes are the derived class of
the **BaseException** class.
Thus **BaseException** class is the topmost class in the hierarchy of exception classes.

Figure 1.4 shows the built-in exceptions viz,

ZeroDivisionError,
NameError, and
TypeError

raised by the Python interpreter in different situations.

```
Python 3.7.0 Shell
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:59:51) [MSC v.1914 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print (50/0)
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    print (50/0)
ZeroDivisionError: division by zero
>>> print (var+40)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    print (var+40)
NameError: name 'var' is not defined
>>> 10+'5'
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    10+'5'
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> |
```

Figure 1.4: Example of built-in exceptions

A programmer can also create custom exceptions to suit one's requirements. These are called **user-defined exceptions**. We will learn how to handle exceptions in the next section.

1.5 RAISING EXCEPTIONS

Each time an error is detected in a program, the **Python interpreter** raises (throws) an exception.

Exception handlers are designed to execute when a specific exception is raised.

Programmers can also forcefully raise exceptions in a program using the **raise** and **assert** statements.

Once an exception is raised, no further statement in the current block of code is executed.

So, raising an exception involves interrupting the normal flow execution of program and jumping to that part of the program (exception handler code) which is written to handle such exceptional situations.

1.5.1 The *raise* Statement

The raise statement can be used to throw an exception.

The syntax of raise statement is:

```
raise exception-name[(optional argument)]
```

The argument is generally a string that is displayed when the exception is raised. For example, when an exception is raised as shown in Figure 1.5, the message “OOPS : An Exception has occurred” is displayed along with a brief description of the error.

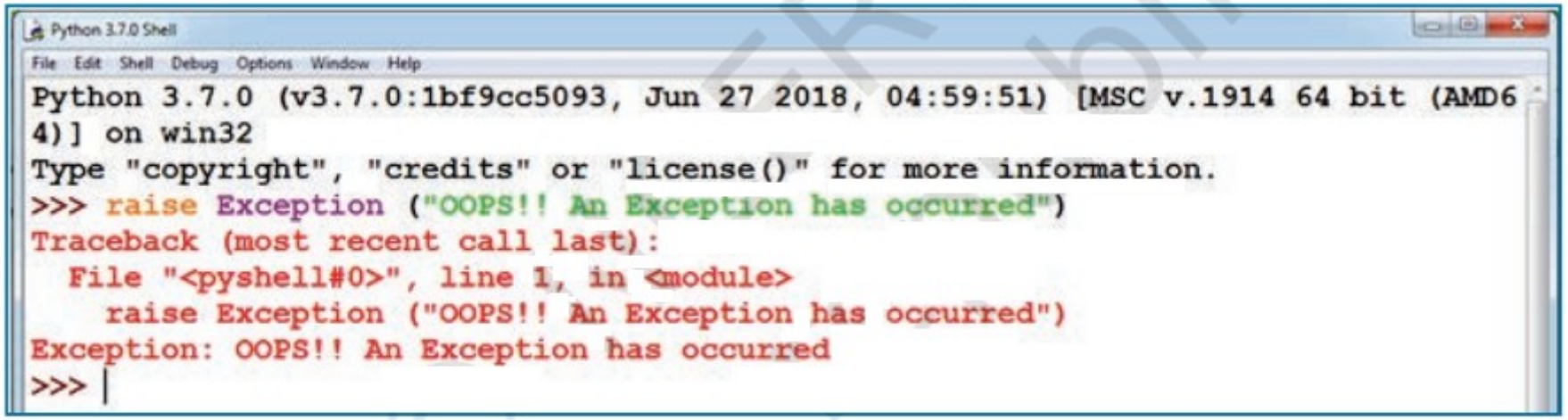


Figure 1.5: Use of the raise statement to throw an exception

The error detected may be **a built-in exception** or may **be a user-defined one**. Consider the example given in Figure 1.6 that uses the raise statement to raise a built-in exception called **IndexError**.

In Figure 1.6, since the value of variable length is greater than the length of the list numbers, an **IndexError** exception will be raised. The statement following the raise statement will not be executed. So the message “NO EXECUTION” will not be displayed in this case.

```
Python 3.7.0 Shell
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:59:51) [MSC v.1914 64 bit (AMD6
4)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> numbers=[40,50,60,70]
>>> length=10
>>> if length>len(numbers):
        raise IndexError
        print ("NO EXECUTION")
else:
        print(length)

Traceback (most recent call last):
  File "<pyshell#7>", line 2, in <module>
    raise IndexError
IndexError
>>> |
```

Figure 1.6: Use of raise statement with built-in exception

1.5.2 The *assert* Statement

An assert statement in Python is used to test an expression in the program code. If the result after testing comes false, then the exception is raised.
This statement is generally used in **the beginning of the function** or after **a function call** to check for valid input.
The syntax for **assert** statement is :

```
assert condition, message
```

`assert` Expression, [arguments]

On encountering an **assert** statement, Python evaluates the expression given immediately after the assert keyword. If this expression is false, an AssertionError exception is raised which can be handled like any other exception. Consider the code given in Program 1-1.

```
1  print("use of assert statement")
2  def negativecheck(number):
3      assert(number>=0), "OOPS... Negative Number"
4      print(number*number)
5  print (negativecheck(100))
6  print (negativecheck(-350))
7
```

Program 1-1 Use of assert statement

```
  File "C:\Users\drsiv\PycharmProjects\csc212PythonProject01\Exception1.py", line 6, in <module>
    print (negativecheck(-350))
  File "C:\Users\drsiv\PycharmProjects\csc212PythonProject01\Exception1.py", line 3, in negativecheck
    assert(number>=0), "OOPS... Negative Number"
AssertionError: OOPS... Negative Number
use of assert statement
10000
None

Process finished with exit code 1
```

Figure 1.7: Output of Program 1-1.

In the code, the assert statement checks for the value of the variable number. In case the number gets a negative value, **AssertionError** will be thrown, and subsequent statements will not be executed.

Hence, on passing a negative value (-350) as an argument, it results in AssertionError and displays the message “OOPS…. Negative Number”. The output of the code is shown in Figure 1.7.

1.6 HANDLING EXCEPTIONS

Each and every exception has to be handled by the programmer to avoid the program from crashing abruptly. This is done by writing additional code in a program to give proper **messages** or **instructions** to the user on encountering an exception. This process is known as ***exception handling***.

1.6.1 Need for Exception Handling

Exception handling is being used not only in Python programming but in most programming languages like C++, Java, Ruby, etc. It is a useful technique that helps in capturing runtime errors and handling them so as to avoid the program getting crashed.

Following are some of the important points regarding exceptions and their handling:

• **Exception Types:** Python categorises exceptions into distinct types so that specific exception handlers (code to handle that particular exception) can be created for each type.

• **Exception handlers separate the main logic of the program from the error detection and correction code.** The segment of code where there is any possibility of error or exception, is placed inside one block. The code to be executed in case the exception has occurred, is placed inside another block. These statements for detection and reporting the exception do not affect the main logic of the program.

• The compiler or interpreter keeps track of the exact position where the error has occurred.

• Exception handling can be done for both user-defined and built-in exceptions

1.6.2 Process of Handling Exception

When an error occurs, Python interpreter creates an object called the exception object. This object contains information about the error like its type, file name and position in the program where the error has occurred.

The object is handed over to the runtime system so that it can find an appropriate code to handle this particular exception.

This process of creating an exception object and handing it over to the runtime system is called ***throwing an exception***.

It is important to note that when an exception occurs while executing a particular program statement, the control jumps to an exception handler, abandoning execution of the remaining program statements.

The runtime system searches the entire program for a block of code, called the **exception handler** that can handle the raised exception.

It first searches for the method in which the error has occurred and the exception has been raised.
If not found, then it searches the method from which this method (in which exception was raised) was called.

This hierarchical search in reverse order continues till the exception handler is found. This entire list of methods is known as **call stack**. When a suitable handler is found in the **call stack**, it is executed by the runtime process.

This process of executing a suitable handler is known as **catching the exception**. If the runtime system is not able to find an appropriate exception after searching all the methods in the call stack, then the program execution stops.

The flowchart in Figure 1.8 describes the exception handling process.

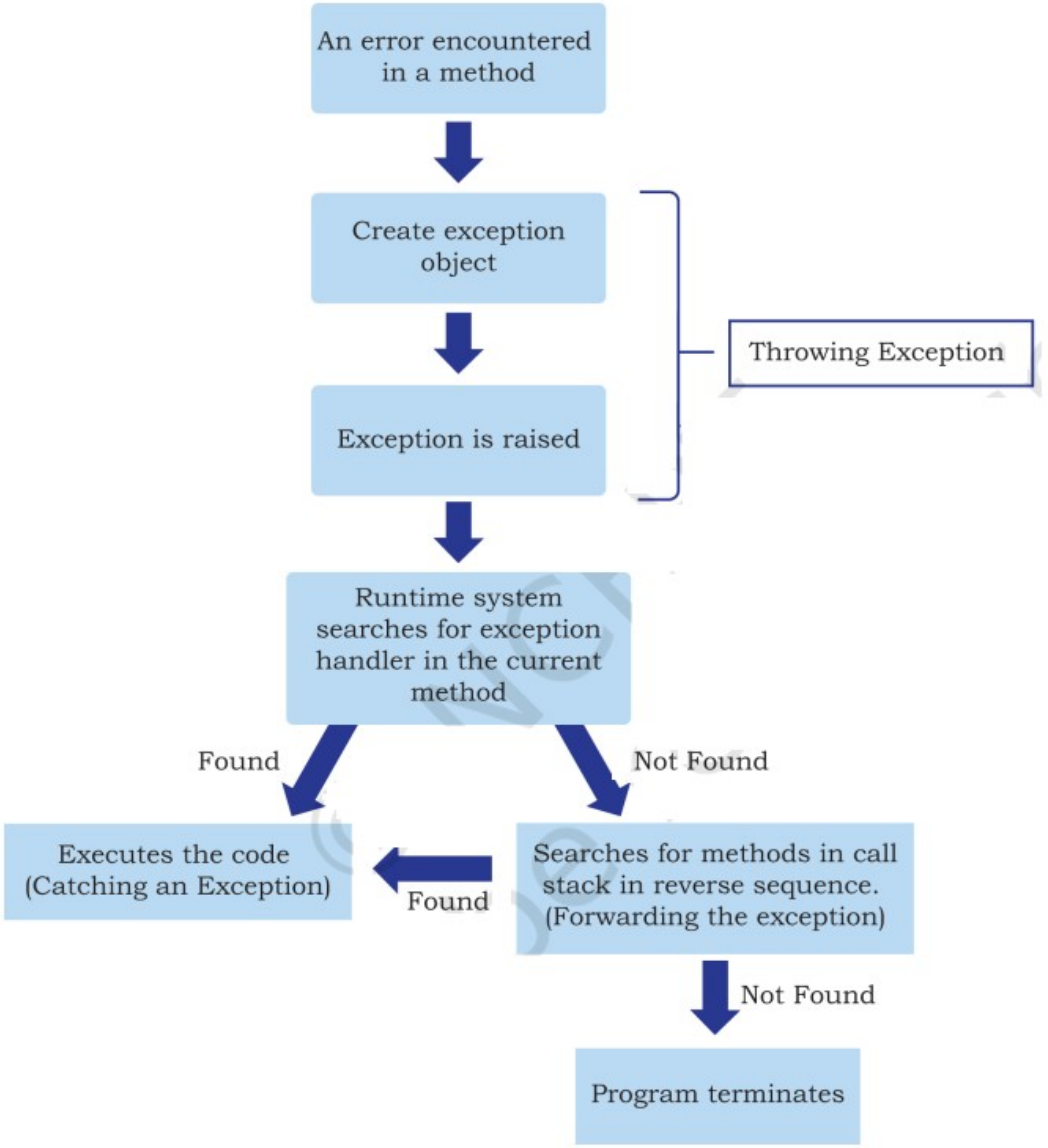


Figure 1.8: Steps of handling exception

1.6.3 Catching Exceptions

An exception is said to be caught when a code that is designed to handle a particular exception is executed. Exceptions, if any, are **caught** in **the try block** and **handled** in **the except block**. While writing or debugging a program, a user might doubt an exception to occur in a particular part of the code. Such suspicious lines of codes are put inside a try block.

Every **try block** is followed by **an except block**.

The appropriate code to handle each of the possible exceptions (in the code inside the try block) are written inside **the except clause**.

While executing the program, if an exception is encountered, further execution of the code inside **the try block** is stopped and the control is transferred to **the except block**.

The syntax of try … except clause is as follows

```
try:
[ program statements where exceptions might occur]
except [exception-name]:
[ code for exception handling if the exception-name error is
encountered]
```

The syntax of *try … except clause* is as follows

```
try:
[ program statements where exceptions might occur]
except [exception-name]:
[ code for exception handling if the exception-name error is
encountered]
```

-The **try block** contains the code that you want to monitor for exceptions. If any exception occurs within this block, Python will jump to the corresponding **except block**, skipping the remaining code in the try block.

- The **except block** is used to specify how to handle exceptions that occur within the try block.

Consider the Program 1-2 given below:

Program 1-2 Using try..except block

```
print ("Practicing for try block")
try:
    print(10/0)
except ZeroDivisionError:
        pass
print ("Welcome to the university")
print ("Shagra")
```

Here program terminated successfully.

```
>>>
    === RESTART: C:/[redacted]
    Practicing for try block
    Welcome to the university
    Shagra
>>>
```

Example 2:

```python
print ("Practicing for try block")
try:
   print(10/0)
except ZeroDivisionError as msg1:
    print("Exception raised : ",msg1)
print ("Welcome to the Unoversity")
print("Shaqra")
```

```
Run:  Exceprion02
Practicing for try block
Exception raised :  division by zero
Welcome to the Unoversity
Shaqra

Process finished with exit code 0
```

In the above code, the first line executed successfully and “Shaqra" will be printed.

When the python interpreter executes the print statement "(10/0)" an exception is raised, and a message given in the print statement will be printed with the name of the exception "division by zero".

After that, the interpreter executes the last statement, and it will print "Welcome to the University.“ and “Shaqra”

If there are **more than one except block** written in **the try block**,
then the try block will execute the except block which is responsible for the code.

Example 3: Try block with more than one except block.

```python
try:
    num1=int(input("enter First Number : "))
    num2=int(input("enter Second Number : "))
    print(num1/num2)
except ZeroDivisionError as msg1:
    print("Can't Divided with Zero : ",msg1)
except ValueError as msg1:
    print("Please Enter integer value : ",msg1)
```

1

```
Run: Exception03
enter First Number : 3
enter Second Number : 4
0.75

Process finished with exit code 0
```

2

```
Run: Exception03
enter First Number : 4
enter Second Number : 0
Can't Divided with Zero :  division by zero

Process finished with exit code 0
```

3

```
Run: Exception03
enter First Number : 4
enter Second Number : q
Please Enter integer value :  invalid literal for int() with base 10: 'q'

Process finished with exit code 0
```

1.6.4 *try...except...else* clause

We can put an optional ***else*** clause along with the ***try...except*** clause.

An except block will be executed only if some exception is raised in the try block.

But if there is no error then none of the except blocks will be executed. In this case, the statements inside the else clause will be executed.

Program 1-3 along with its output explains the use of else block with the try...except block.

Program 1-3 Use of else clause

```
1  try:
2      num1=int(input("enter First Number : "))
3      num2=int(input("enter Second Number : "))
4      quotient= num1/num2
5  except ZeroDivisionError as msg1:
6      print("Can't Divided with Zero : ",msg1)
7  except ValueError as msg1:
8      print("Please Enter integer value : ",msg1)
9  else:
10     print ("The result of division operation is ", quotient)
```

Run: Exception03

```
enter First Number : 2
enter Second Number : 1
The result of division operation is  2.0

Process finished with exit code 0
```

1.7 FINALLY CLAUSE

The ***try*** statement in Python can also have an optional ***finally*** clause.

The statements inside **the finally block** are always executed regardless of whether an exception has occurred in the try block or not.

It is a common practice to use finally clause while working with files to ensure that the file object is closed. If used, finally should always be placed at the end of try clause, after all **except blocks** and the **else block**.

In the following program, the message "OVER AND OUT" will be displayed irrespective of whether an exception is raised or not.

```python
try:
    num1=int(input("enter First Number : "))
    num2=int(input("enter Second Number : "))
    quotient= num1/num2
    print("Devision Performed Successfully")
except ZeroDivisionError as msg1:
    print("Can't Divided with Zero : ",msg1)
except ValueError as msg1:
    print("Please Enter integer value : ",msg1)
else:
    print ("The result of division operation is ", quotient)
finally:
    print("OVER and OUT")
```

Run: Exception04

```
enter First Number : 4
enter Second Number : 3
Devision Performed Successfully
The result of division operation is  1.3333333333333333
OVER and OUT

Process finished with exit code 0
```

1.8 Nested *try-except* block

If a try block is written inside another try block, then it is called as nested.

The code having higher risk must be defined in the innermost try block.

If there is any exception raised by the innermost try block, then it will be handled by the innermost except block.

If the innermost except block is not able to handle the error then except block defined outside the inner try block will handle the exception.

```
try:
    print("Outer try block")
    try:
        print("Inner try block")
        print(22/0)
    except ZeroDivisionError:
        print("Except block of Inner try block")
    finally:
        print("Finally block of Inner try block")
except:
    print ("Except block of Outer try block")
finally:
        print("Finally block of Outer try block")
```

```
>>>
    === RESTART: C:/Users/Ohud/AppData/
    Outer try block
    Inner try block
    Except block of Inner try block
    Finally block of Inner try block
    Finally block of Outer try block
>>>
```

Example 4: Implementation of the nested try block

A single except block can be used to **handle multiple exceptions**. Consider the given an example:

Example 5: Single except block handling multiple exceptions.

```python
try:
    num1=int(input("Enter First Number : "))
    num2=int(input("Enter Second Number : "))
    print(num1/num2)
except (ZeroDivisionError, ValueError) as msg1:
    print("Invalid Number & Error is : ",msg1)
```

try

Run: Exception06

```
Enter First Number : 4
Enter Second Number : 0
Invalid Number & Error is :  division by zero

Process finished with exit code 0
```

Run: Exception06

```
Enter First Number : 4
Enter Second Number : two
Invalid Number & Error is :  invalid literal for int() with base 10: 'two'

Process finished with exit code 0
```

In case of different types of error, we can use **a default except for block**. This block is generally used to display normal error messages. The default except block must be **the last block of all except blocks.**

Example 06: Implementation of default except for block.

```python
try:
    num1=int(input("Enter First Number : "))
    num2=int(input("Enter Second Number : "))
    print(num1/num2)
except (ZeroDivisionError):
    print("ZeroDevissionError : Can't divide with zero")
except:
    print("Default Except : Plz provide valid input only")
```

```
Run: Exception06
Enter First Number : 5
Enter Second Number : ten
Default Except : Plz provide valid input only

Process finished with exit code 0
```

```
Run: Exception06
Enter First Number : 5
Enter Second Number : 0
ZeroDevissionError : Can't divide with zero

Process finished with exit code 0
```

- **User-defined exceptions**, also known as **custom exceptions**, are exceptions that you create in Python to handle specific error conditions or exceptional situations in your code.

Example 10: Implementation of user-defined exceptions.

```
class EligibleForEntrance (Exception ) :
        def __init__(self,arg1) :
                self.msg1=arg1

class NotEligible (Exception) :
        def __init__ (self,arg1) :
                self.msg1=arg1

percentage=int (input ("Enter Your PCM Marks percentage"))
if percentage>50:
        raise EligibleForEntrance("You can apply for the entrance examination")
else:
        raise NotEligible("You cant apply for the entrance examination")
```

Ln: 5 Col: 31

```
Enter Your PCM Marks percentage 88
__main__.EligibleForEntrance: You can apply for the entrance examination
>>>
================ RESTART: D:/python/IGNOU/Unit 14/Example 10.py ================
Enter Your PCM Marks percentage 44
__main__.NotEligible: You can't not apply for the entrance examination
```

A *raise* statement is used to raise the statement.

Example 11: Program to raise an exception.

```
class TooYoungException(Exception):
    def __init__(self,arg):
        self.msg=arg

class TooOldException(Exception):
    def __init__(self,arg):
        self.msg=arg

age=int(input("Enter Age:"))
if age>60:
    raise TooYoungException("Plz wait some more time you will get best match soon!!!")
elif age<18:
    raise TooOldException("Your age already crossed marriage age...no chance of getting marriage")
else:
    print("You will get match details soon by email!!!")
```

Ln: 8 Col: 0

```
============================ RESTART: D:/python/IGNOU/Unit 14/Example 11.py ============================
Enter Age:55
Plz wait some more time you will get best match soon!!!
>>>
============================ RESTART: D:/python/IGNOU/Unit 14/Example 11.py ============================
Enter Age:33
You will get match details soon by email!!!
>>>
```